



MIDDLE EAST TECHNICAL UNIVERSITY
NORTHERN CYPRUS CAMPUS

DEPARTMENT
OF
COMPUTER ENGINEERING

CNG 351
Data Management and File Structures
Assignment 5
(4% of actual grade)

DUE DATE: 5 January, Sunday, 2025, 23:55 (Cyprus Time)

PURPOSE

This assignment aims to help you revise the last part of the course which is file structures and indexing. You will mainly have questions that will help you revise the following topics: Disk Storage, Basic File Structures and Hashing, and Indexing Structures for Files.

IMPORTANT RULES

1. Please make sure that your solutions are clearly explained.
2. Please make sure that your report is readable.
3. When you create an index, please clearly show all the steps.
4. Create a PDF file for your solution, and upload only one PDF file to ODTUClass, one team member is enough to upload the solution.
5. Your PDF file can include scans/images of handwritten solutions.
6. Assignments will be completed by the team that was formed for the previous assignment.
7. Please submit a report that includes a cover page with the team details including their names/surnames and also student IDs.

GRADING

This assignment has four questions and the overall grading will be as follows:

1. Question 1 (Hash Index): 25 points;
2. Question 2 (Extendible Hash Index): 30 points;
3. Question 3 (B+ Tree): 15 points;
4. Question 4 (Clustering and Secondary Index): 25 points;
5. Report: 5 points. A good report means type written, complete (every section fulfilled), with clear explanations in English (where relevant), and submitted via ODTUCLASS as one combined PDF document. The first page of the report must be a title page which should clearly state team details and assignment number. Each use case should also include Assumptions clearly written. The footer of all subsequent pages should be numbered in the format x of y (eg 2 of 6), etc.

Question 1 (25 pts)

Consider the following relation that includes information about player details in our velocity game. This table keeps track of the player id, nickname of the player, elo score and credits.

player_id	nickname	elo_score	credits
2981	snakeserdar	1323	630.08
5252	kharp haz	959	6326.47
3153	SametBaba	1046	2420.29
2424	DominicToretto	1520	9910.53
7045	xKralTR	1092	17009.64
1351	TylerDurden	1173	18594.54
2147	Sp33d4dd1ct	1428	3704.63

Table 1: Player Relation.

- Imagine that you are using a Hash file at the back end to store the data given in this relation. Assume that the hash function h of this file takes $\text{player_id} \bmod 5$. For example, Let's look at the first tuple in the relation. $h(2981) \bmod 5 = 1$. Assume that all hash values fit in one block, and only one record can fit in one block. Organise the data in this given relation in such a Hash file and show the resulting structure. Please note that for collision, the system uses a chained overflow approach. Please clearly show how your data is inserted and all the details of your calculations. You need to clearly show all your decisions, explain and justify your steps.
- Imagine that you are using another Hash file at the back end to store the data given in this relation. This time, assume that the hash function h takes both the player_id and also year elo_score and calculates $\bmod 5$. For example, let's look at the first tuple in the relation. $h(2981+1323) \bmod 5 = 4$. Assume that all hash values fit in one block, and only one record can fit in one block. Organise the data in this given relation in such Hash file and show the resulting structure. Please note that for collision, the system uses chained overflow approach. Please clearly show how your data is inserted and all the details of your calculations. You need to clearly show all your decisions.
- Now, look at the file organization resulting from the above two questions. (1) Which hash function would you prefer to use and (2) why? Please clearly explain why you think one is better than the other. It is not enough to say that one is better than the other; you need to justify your answer.

Question 2 (30 pts)

Imagine that your table in Question (1) now has more data. This time imagine that you are using an Extensible hashing to insert data. Load the records from

player id: 4062, 4066, 2617, 4069, 4070, 4074, 2626, 4078, 4082, 2633, 4086, 4088, 2610 (you only need to load these records and not the ones given in the previous question). Show the directory structure at each step, and the global and local depths. Use the hash function $h(K) = K \bmod 29$. Imagine that each bucket is one disk block and holds 3 records. Please make sure that the depth details of all the buckets are given and that your step-by-step insertion is clearly shown. **Please use leftmost bits to put data into blocks!**

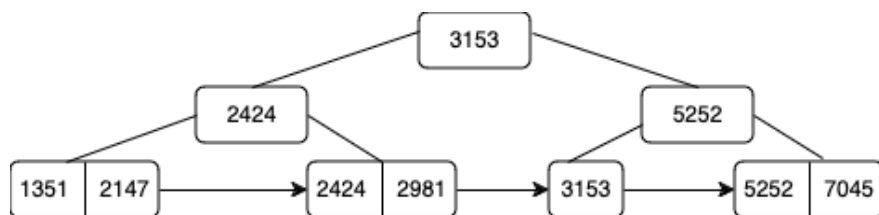
Example:

Assume the value is 001111.

You need to check from the leftmost bits $\rightarrow$ 001110.

Question 3 (15 pts)

Assume that data in the Table given in Question 1 (see Table 1) is inserted in the given order into a B+ tree. This B+ tree only stores the player_id. Therefore, the following tree structure is constructed. Please note that the maximum degree is 3 in this tree structure. Answer the following questions based on this given B+ tree.



- Show the nodes that will be visited to return the result of this query. You need to highlight them on the tree and label the nodes to show the order of visit:

```
SELECT * FROM Player WHERE player_id='5252'
```

- Show the structure of the tree after the following query is executed:

```
DELETE FROM Player WHERE player_id='3153'
```

Question 4 (25 pts)

Consider the relation given below, which stores details of cars participating in races. Each race has a unique id and more than one player can participate in a given race and each car has a unique name in this database. Therefore, car name is a secondary key. Based on this table, answer the following questions accordingly. Assume each block can contain 4 records, and the original data file is sorted by the primary key race_ID.

race_id	player_id	car_name	duration	position
216	2981	lokmata1	5.31	5
431	2424	FamilyFirst	4.12	2
432	2147	GottaGoF4st	5.13	8
482	3153	DocHudson	5.01	3
483	1351	doNotT4lk	4.57	2
484	2981	lokmata2	4.56	1
526	2424	FamilyFirstYou	0.55	1
527	3459	McQueen	3.58	7
528	1351	SoapMachine	3.12	3
574	2981	lokmata3	4.32	1
848	1351	SoapMachine2	4.14	2

1. Show the database structure when the given table is indexed using a clustering index with player_id.
2. Show the database structure when the given table is indexed using a secondary key car_name.